

BRAC University

Department of Computer Science and Engineering

CSE111: Programming Language II

Examination: *Theory Assignment #2*

Semester: *Summer 2022*

Date: ____ / ____ / 2022

Deadline: 22 August, 2022

ID: _____	Name: (Please write in CAPITAL LETTERS)	Section: 42
--------------	--	-----------------------

1. A lot of online food delivery companies have been opened in Bangladesh just within a few years. It has become challenging for the country to keep track of these companies. Therefore, the technical department of the Ministry of Food, Bangladesh is looking to hire new amazing programmers who are excellent in Object Oriented Programming, particularly in Python because of its growing global popularity.

For the selection purpose, only those applicants who could reverse engineer and design the required classes in Python from the given driver code and output will be called for an interview.

To help the applicants to understand the requirements better, the following class diagrams are given that must be followed while reverse engineering the given driver code below.

Restaurant (R)	DeliveryCompany (DC)	MinistryOfFood (MOF)
+ <u>count</u>: int + id: int + name: str	+ <u>count</u>: int + id: int + name: str + platform: str + restaurants: list + mof: MinistryOfFood	+ <u>instances</u>: dict + <u>target</u>: str + country: str + restaurants: list + delivery_companies: list
- <u>__init__</u>: None - <u>__str__</u>: str	- <u>__init__</u>: None - <u>__str__</u>: str + register: tuple + show_info: None	- <u>__init__</u>: None - <u>__str__</u>: str - <u>__class_getitem__</u>: MOF - <u>__getitem__</u>: DC R + <u>create_instance</u>: MOF + <u>set_target</u>: None + add_restaurant: None + add_delivery_companies: None + print_summary: None

Legends:

Underline: Class Properties
Italic: Methods/ Functions
+: Public Properties
-: Private Properties

Driver Code:

```
# Write your class code here

mof_bd = MinistryOfFood.create_instance("BD")
sultans_dine = Restaurant("Sultan's Dine")
foodpanda = DeliveryCompany("Foodpanda")
chillox = Restaurant("Chillox", foodpanda)
print(MinistryOfFood["BD"] == mof_bd)
hungry_naki = DeliveryCompany("Hungry Naki")
kfc = Restaurant("Kentucky Fried Chicken")
kfc_delivery = DeliveryCompany("KFC@Door", "Offline")
hungry_naki.register(*foodpanda.register(sultans_dine, chillox, kfc)[:2])
mof_bd.print_summary()
print("/", mof_bd["restaurant", "chillox"])
print("\\", mof_bd["delivery Company", "hungry Naki"])
mof_bd["delivery company", "hungry naki"].show_info()
```

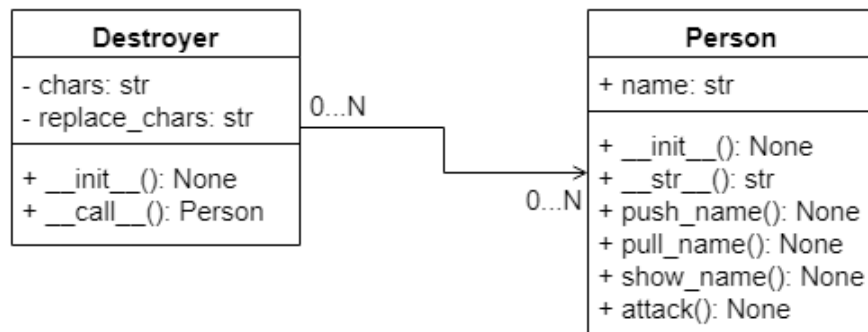
Output:

```
True
1: Chillox already registered under Foodpanda
=====
Ministry of Food | BD
Total restaurants: 3
  - 0: Sultan's Dine
  - 1: Chillox
  - 2: Kentucky Fried Chicken
Total delivery companies: 3
  - 0: Foodpanda [Online] {3 registered restaurants}
  - 1: Hungry Naki [Online] {2 registered restaurants}
  - 2: KFC@Door [Offline] {0 registered restaurants}
=====

/ 1: Chillox
\ 1: Hungry Naki [Online] {2 registered restaurants}
Hungry Naki is an Online food delivery company.
Registered under `Ministry of Food (BD)` with ID: 1.
Total registered restaurants: 2
  - 0: Sultan's Dine
  - 1: Chillox
```

2. Design a Destroyer and Person class so the following driver code generates the given output upon execution.

A Person has only `name` as its property which is publicly accessible and has a few public functions. All the properties (or variables) of a Destroyer object, on the other hand, are private. The variables are **chars** - a string containing all the characters that a particular **Destroyer** object will remove from a **Person's name** and **replace_chars** - a string with which each character in chars will be replaced from a **Person's name**. You may use the class diagram given below.



Driver Code:

```

john = Person("John")
alberta = Person("Alberta")
print(john, alberta)
v_kill = Destroyer("aeiou")
v_kill(john)
print(john, alberta)
alpha_kill = Destroyer("".join(chr(i) for i in range(65, 91)), "")
print(john, alberta)
alpha_kill(john)
print(john, alberta)
alberta.push_name(john)
print(john, alberta)
caitlin = Person("Caitlin")
john.attack(caitlin)
print(caitlin)
v_kill(caitlin).show_name()
alberta.show_name()
  
```

Output:

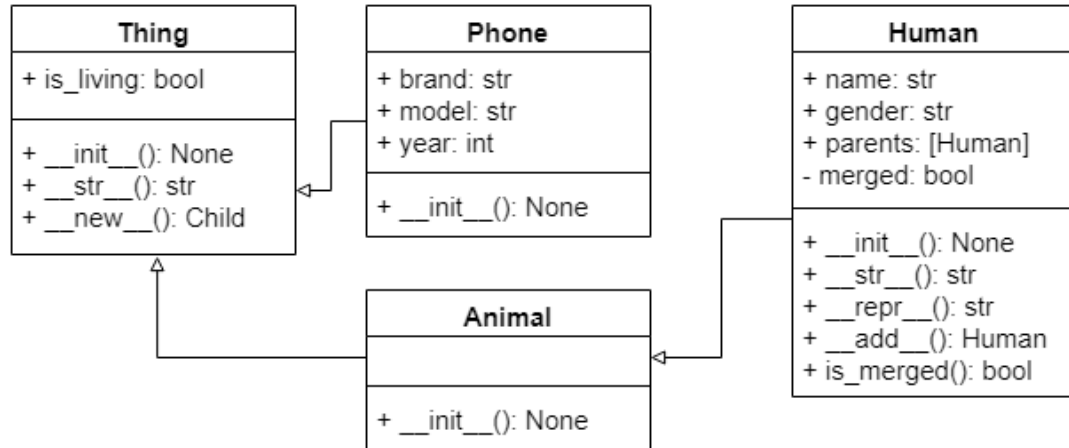
```

John Alberta
Destroyed: 'o' from J#hn's name.
J#hn Alberta
J#hn Alberta
Destroyed: 'h', 'j', 'n' from #'s name.
# Alberta
Alberta Alberta
Ciin
Destroyed: 'i' from C##n's name.
Hello, I am C##n
Hello, I am Alberta
  
```

- Given a **Thing** class, you need to reverse engineer from the given driver code and output to design and create the **Phone**, **Animal**, and **Human** classes. You may take help from the given class diagram to understand the significance of inheritance among the classes.

The gender of an offspring (addition of two Human objects) is determined by the result of the equation, $\text{sum_of_length_of_parents_name} \% 2$. If the result of the equation is 0 then the gender of the offspring is "Female" otherwise, "Male".

For naming the offspring, the first half is taken from the first parent and the second half from the second parent for both the first and last names of the offspring.



Driver Code:

```

clark = Human("Clark Kent", "Male")
lois = Human("Lois Lane", "Female")
iPhone_13 = Phone("Apple", "iPhone 13")
iPhone_111 = Phone("Apple", "iPhone 111 Max Pro", 2030)
thing = Thing()

print(clark)
print(iPhone_13)
jordan = clark + lois
print(jordan)
sarah = Human("Sarah Cushing", "Female")
sardan = jordan + sarah
print(sarah)
print(sardan)
print(clark.is_merged, sarah.is_merged, sardan.is_merged)
clark.is_merged = True
print(clark.is_merged, sarah.is_merged, sardan.is_merged)
print(repr(sarah))
  
```

Output:

```

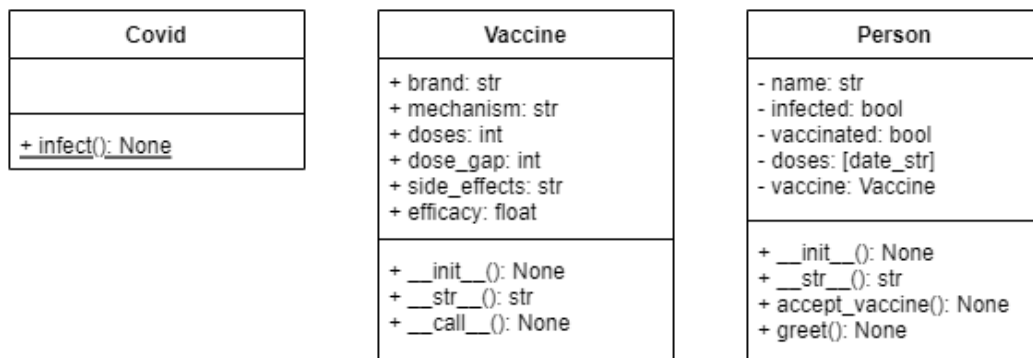
Thing is an abstract class. Cannot create objects.
Living thing.
Non-Living thing.
Clis Kene is a male child of [Clark Kent, Lois Lane]
Living thing.
Clrah Kehing is a female child of [Clis Kene, Sarah Cushing]
False False True
False False True
Sarah Cushing
  
```

4. The Covid-19 Pandemic - a pandemic caused by the Coronavirus - started in 2019 on our earth, Earth-111. The virus is very well known for its mutations and ability to spread. After the felling of TVA - Time Variance Authority, **Covid** was successful to spread throughout the multiverse to infect more hosts.

Recently, you were informed about its existence in the universe of Hobbits - which started from the Shire city. To turn the entire Hobbits universe into a pandemic, it just needed to **infect** one significant individual and found **Frodo** Baggins - nephew of Bilbo Baggins. Because the virus can be transmitted from just one person to everyone else while a meetup - for example: during **greeting** each other, the virus has been spread to other parts of the Hobbits universe very quickly.

The wizards of Hobbits himself have been infected and Thorin Oakenshield - the king of Dwarves - is looking for help from external universes. Since Earth-111 is the first to develop vaccines that have been proved to slow down the spread and minimize the effects significantly, Oakenshield is planning to import vaccines from our earth. Also considering the rate of spreading of the virus, Oakenshield has asked you to build a computer-based system to keep track of the infected individuals and their vaccine status as there might be a lack of human resources to do this task.

Now, since you are learning Object Oriented Programming in Python, to test if you are eligible for the role, you are given the following driver code, and the class diagram to design the **Covid**, **Vaccine**, and **Person** class in Python so that the following expected output is generated.



Although the calendar of the Hobbits universe is similar to that of ours, there are slight changes. Unlike our months, every month (of 12) in the Hobbits universe has 31 days if it's an odd month otherwise 30 days so, they have a total of 366 days in every year - yes, there are no leaps or non-leap years. However, you do not have to worry about this as a global (lambda) function to convert a date (string) to days (integer) has been provided to you to use in classes to convert any date to a number of days.

Driver Code:

```
days = lambda d : sum([
    (
        int(x) +
        (30 * int(y) + int(y) // 2) +
        366 * (2000 + int(z[-2:]))
    ) for x, y, z in [d.split("/")]
])
```

```

pfizer = Vaccine("Pfizer", "mRNA", 2, 21, "Allergies & Anaphylaxis",
0.95)
moderna = Vaccine("Moderna", "mRNA", 2, 28, "Facial Paralysis", 0.94)
astra_zeneca = Vaccine("Astra-Zeneca Oxford", "Adenovirus Viral vector",
2, 12, "Blood-clots, pulmonary embolism, etc",
0.7)
frodo = Person("Frodo")
bilbo = Person("Bilbo")
gandalf = Person("Gandalf")
gollum = Person("Gollum")

pfizer(frodo, "5/8/2022")
Covid.infect(frodo)
pfizer(frodo, "5/8/2022")
pfizer(frodo, "10/8/2022")
pfizer(frodo, "26/8/2022")
pfizer(frodo, "26/8/2022")
bilbo.infected = True
print("Bilbo infected:", bilbo.infected)
frodo.greet(bilbo)
print("Bilbo infected:", bilbo.infected)
print("Gandalf infected:", gandalf.infected)
gandalf.greet(frodo, gollum)
print(gandalf.infected, gollum.infected, frodo.infected)
moderna(gollum, "27/8/22")
pfizer(gollum, "20/9/22")
moderna(gollum, "20/9/22")
moderna(gollum, "29/9/22")
print(pfizer)

```

Output:

```

Frodo is not infected.
Frodo is vaccinated with 1 Pfizer dose(s).
Dear Frodo, please come back after 16 days.
Frodo is vaccinated with 2 Pfizer dose(s).
Dear Frodo, your 2 doses for Pfizer has been completed.
Virus key mismatched
Bilbo infected: False
Bilbo infected: True
Gandalf infected: False
True True True
Gollum is vaccinated with 1 Moderna dose(s).
Gollum is vaccinated with Moderna hence, cannot take Pfizer vaccine.
Dear Gollum, please come back after 5 days.
Gollum is vaccinated with 2 Moderna dose(s).
Brand: Pfizer
Mechanism: mRNA
Doses: 2
Dose gap: 21
Side effects: Allergies & Anaphylaxis
Efficacy: 95%

```

5. Noticing the neverending list of browsers available on the market - which is still growing, the Browser Grants Commission (BGC) passed an order to centralize all the browsers under a single domain Browser in order to keep the user's privacy secured and provide world-class services to all users.

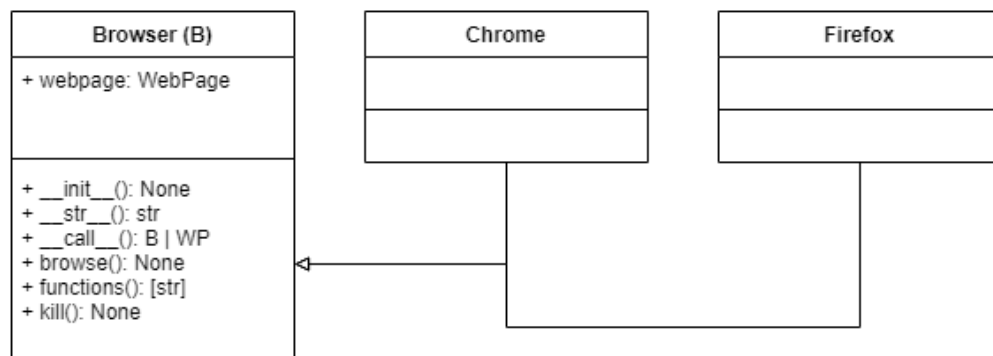
Your friend - a Junior Software Engineer at BGC - was given the task to design such a system. The design was at its initial stage of development and your friend was still developing when the load shedding crisis begin. Because her workplace used to have pretty stable electricity, she did not think of having any backup power - such as an Uninterruptible Power Supply (UPS). Her computer was going through the irresistible Windows update when this particular load shedding struck and her system got suddenly shut down. Although she had all her project files saved safely before updating - unfortunately, it was stored only in the local machine and not in any cloud storage. Lucky her that the system was not corrupted and it was successfully booted when she got the power back. However, not so lucky, project files: the class diagram, code, and output has been corrupted.

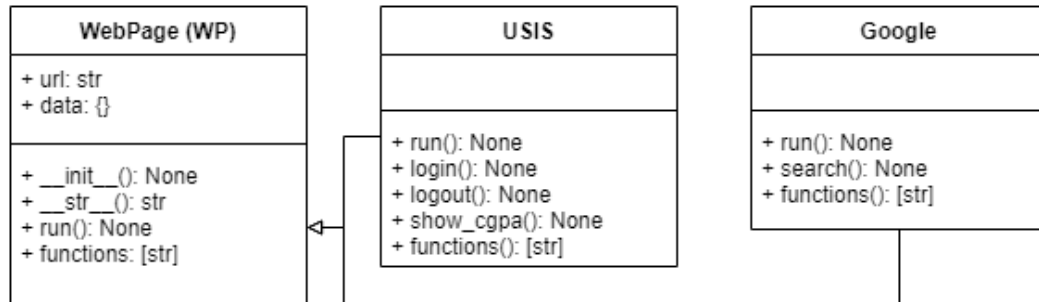
The class diagrams of Chrome and Firefox lost all the information. Moreover, it can be seen that when we create a Chrome object, we are getting a Firefox object. This was the first project given to her, she is very much worried about it and cannot think straight at the moment so, she needs your help.

Given the Browser and WebPage classes, she asked you to design the Chrome, Firefox, USIS, and Google classes to generate the given expected output if the given driver code below has been executed. The class diagram has been provided to you, for your convenience. Furthermore, to make it less burdensome for you, you do not need to figure out what was in the Chrome and Firefox class diagrams, you just need to design these two classes so that the following output is generated and she can figure out what is the issue and fix those later.

Besides, you also need to design the USIS and Google classes so, you need to know how these two classes work - at least the basics. For USIS, you only need to know how the CGPAs are calculated. To calculate the CGPA, you first need to enumerate the alphabets (case-insensitive) from 1 for a (or A) to 26 for z (or Z) and find the summation of all the alphabets from a student's email address before and excluding the '@' character. Also, you need to count how many alphabets are available since you need to calculate the weighted average of the sum where each alphabet has a weight of 26. Finally, convert the calculated average to a 4-point CGPA.

The search function is the only function that requires explanations in the Google class. The number of queries is determined by the summation of all the ASCII values of each character in the query times a million. The time required to generate the result is calculated by dividing 1 by the summation of ASCIIs of the characters, however, the summation for time has a starting offset of 100.





Given classes

```

class Browser:
    def __init__(self):
        self.webpage = None

    def browse(self, webpage, input_data = None):
        self.webpage = webpage
        print(f"{self} is now browsing {webpage}")
        webpage.run(input_data)

    def functions(self):
        if self.webpage:
            return self.webpage.functions()
        return []

    def kill(self):
        self.webpage = None

    def __call__(self):
        if self.webpage:
            return self.webpage
        return self

    def __str__(self):
        return self.__class__.__name__

class WebPage:
    def __init__(self, url):
        self.url = url
        self.data = {}

    def run(self, input_data=None):
        print("Yet to be implemented")

    def functions(self):
        return ["Yet to be implemented"]

    def __str__(self):
        return self.url
  
```

Driver Code:

```
# Write your code here

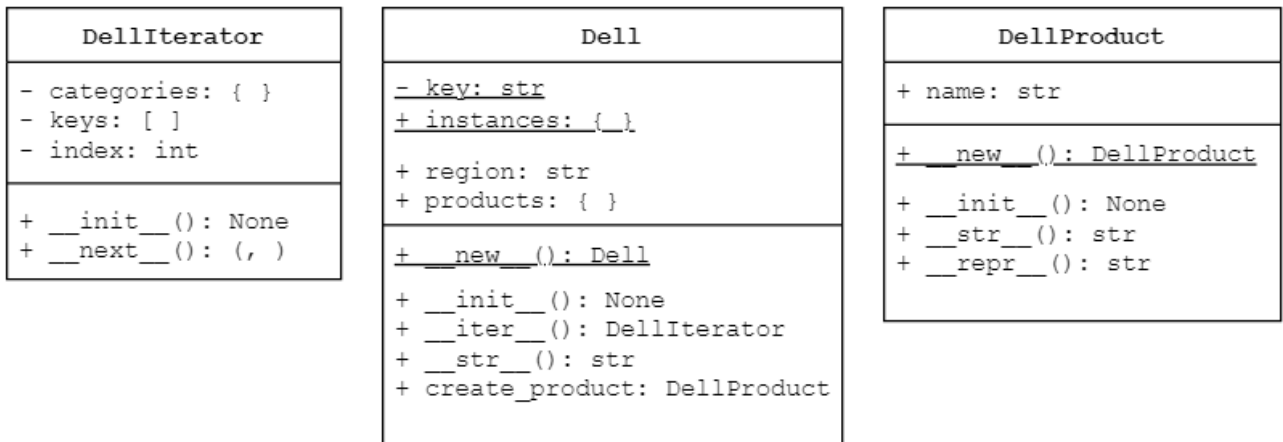
chrome = Chrome()
print(chrome)
usis = USIS("https://usis.bracu.ac.bd/academia/")
google = Google("https://www.google.com/")
john_login = "john@g.bracu.ac.bd", "assignment2"
zyq_login = "zyq@g.bracu.ac.bd", "assignment2"
chrome().browse(usis, john_login)
options = chrome().functions()
print("Current options:", options, sep="\n")
chrome().show_cgpa()
chrome().login(*zyq_login)
chrome().logout()
chrome().login(*zyq_login)
chrome().show_cgpa()
chrome().logout()
chrome().logout()
chrome.kill()
print(chrome())
print(chrome().functions())
chrome().browse(google)
print("Options:", chrome().functions())
chrome().search("How to create an abstract class in Python?")
```

Output:

```
Firefox
Firefox is now browsing https://usis.bracu.ac.bd/academia/
=====
Logging with...
Email: john@g.bracu.ac.bd
Password: assignment2
=====
Current options:
['show_cgpa()', 'login(email: str, passwrod: str)', 'logout()']
Your CGPA is: 1.81
john@g.bracu.ac.bd needs to logout first!
john@g.bracu.ac.bd logged out successfully.
=====
Logging with...
Email: zyq@g.bracu.ac.bd
Password: assignment2
=====
Your CGPA is: 3.49
zyq@g.bracu.ac.bd logged out successfully.
You need to login first.
Firefox
[]
Firefox is now browsing https://www.google.com/
Found 0 results at 0.01 seconds.
Options: ['search(query: str)']
Found 3894 million results at 0.00025 seconds.
```

6. From the given driver code and the expected output, design the DellIterator, Dell, and DellProducts classes. You may use the class diagram given below to understand the required properties and functions of the classes.

While designing the Dell class, you need to be careful about there can only be one instance of Dell in a particular region. Moreover, an object of a Dell class should be able to iterate - you may use the DellIterator class here. Furthermore, a DellProduct object can only be created with a secret key that has been stored in the Dell class, for example, "D3771>R01)U(7".



Driver Code:

```
dell_bd = Dell("Bangladesh")
ph24 = DellProduct("Dell Monitor")
print(ph24)
ph24 = dell_bd.create_product("Monitor", "PH2419H")
print(ph24)
dellbd = Dell("Bangladesh")
print(dellbd)

for category, products in dellbd:
    print(category, products)

cart = {"Mouse": ["WM126", "MS111", "WM527"],
        "Monitor": ["P2720D", "U4021QW", "U2720Q"],
        "PC": ["XPS 8940", "Optiplex 5490", "Inspiron 3891"],
        "Laptop": ["XPS 15", "Inspiron 3000", "Alienware m17 R4"],
        "Keyboard": ["KB216", "KM713", "7VHY1"]}

for k, v in cart.items():
    for _ in v:
        dellbd.create_product(k, _)

print(dell_bd)
for category, products in dellbd:
    print(category, products)
```

Output:

```
Key missing! Failed to create the product.
None
PH2419H
=====
Dell Bangladesh
1 categories | 1 products
=====
Monitor [PH2419H]
=====
Dell Bangladesh
5 categories | 16 products
=====
Monitor [PH2419H, P2720D, U4021QW, U2720Q]
Mouse [WM126, MS111, WM527]
PC [XPS 8940, Optiplex 5490, Inspiron 3891]
Laptop [XPS 15, Inspiron 3000, Alienware m17 R4]
Keyboard [KB216, KM713, 7VHY1]
```

7. Trace the following code:

```
class Natural:
    def __init__(self, val):
        self.val = val

    @property
    def val(self):
        return self.__val

    @val.setter
    def val(self, val):
        if val < 0:
            val *= -1
        self.__val = val

    def __repr__(self):
        return f"{self.val}"
```

Driver code

```
v = [-1, 2, -4, 8, -16, -32, 64, -128, 512, 1024]
r = list(range(1, 10, 2))
r = r + list(range(0, 10, 2))
i = 0
while i < len(r):
    _ = r[i]
    n = Natural(v[_])
    print(n)
    i += 1
```

8. Trace the following code:

<pre> class A: def __init__(self): self.v1 = "A" def method1(self): return self.v1 def method2(self): return self.v2 def method3(self): return self.v3 class B(A): def __init__(self): super().__init__() self.v1 = "B" self.v2 = "B2" self.v3 = "B3" def method1(self): return "Method 1" class C(B): def __init__(self): self.v1 = "C" super().__init__() self.v3 = "C3" def method1(self): return A.method1(self) </pre>	<pre> # Driver Code c = C() print(c.method1()) print(c.method3()) print(isinstance(c, A)) print(c.method2()) b = B() print(b.method3()) </pre>
--	---

Difficulty Chart:

Question	Difficulty Level	Difficulty Bar
Question 1	4	
Question 2	2	
Question 3	4	
Question 4	3	
Question 5	5	
Question 6	4	

** Submit at least one programming and one tracing problem **
~ Best of Luck ~